4.4 Instrumentation

Sue has just finished her first draft of a function to solve a Sudoku puzzle. It seems fast, but she is unsure how fast it really is. How do you measure performance on a machine that can do billions of instructions per second? To get to the bottom of this, she introduces counters in key parts of her program to measure how many times certain operations are performed.

Objectives

By the end of this class, you will be able to:

- Instrument a function to determine its performance characteristics.

Prerequisites

Before reading this section, please make sure you are able to:

- Search for a value in an array (Chapter 3.1).
- Look up a value in an array (Chapter 3.1).
- Declare an array to solve a problem (Chapter 3.0).
- Write a loop to traverse an array (Chapter 3.0).
- Predict the output of a code fragment containing an array (Chapter 3.0).

Overview

One of the most important characteristics of a sorting or search algorithm is how fast it accomplishes its task. There are several ways we can measure speed: the number of steps it takes, the number of times an expensive operation is performed, or the elapsed time.

To measure the number of steps the operation takes, we need to add a counter to the code. This counter will increment with every step, resulting in one metric of the speed of the algorithm. We call the process of adding a counter "**instrumenting**." Instrumenting is the process of adding code to an existing function which does not change the functionality of the task being measured. Instead, the instrumenting code measures how the task was performed. For example, if consider the following code determining if two strings are equal:

```
/*************************************************
 * IS EQUAL
 * Simple function to tell if two strings are equal
 *************************************************/
bool isEqual(const char * text1, const char * text2)
{
   while (*text1 == *text2 && *text2)
   {
      text1++;
      text2++;
   }

   return *text1 == *text2;
}
```

This same code can be instrumented by adding a counter keeping track of the number of characters compared:

```
/*********************************************
 * IS EQUAL
 * Same function as above except it will keep track
 * of how many characters are looked at to determine
 * if the two strings are equal. This function is
 * instrumented
 *********************************************/
bool isEqual(const char * text1, const char * text2)
{
   int numCompares = 1; // keeps track of the number of compares

   while (*text1 == *text2 && *text2)
   {
      text1++;
      text2++;
      numCompares++;  // with every compare (in the while loop), add one!
   }

   cout << "It took " << numCompares << " compares\n";
   return *text1 == *text2;
}
```

Write a program to manage your personal finances for a month. This program will ask you for your budget income and expenditures, then for how much you actually made and spent. The program will then display a report of whether you are on target to meet your financial goals

Sue's Tips

It is a good idea to put your instrumentation code in #ifdefs so you can easily remove it before sending the code to the customer. The most convenient way to do that is with the following construct:

```
#ifdef DEBUG
#define Debug(x) x
#else
#define Debug(x)
#endif
```

Observe how everything in the parentheses is included in the code if `DEBUG` is defined, while it is removed when it is not. With this code, we would say something like:

```
Debug(numCompare++);
```

Again, if `DEBUG` were defined, the `numCompare++;` would appear in the code. If it were not defined, then an empty semi-colon would appear instead.

Example 4.4 – Instrument the Bubble Sort

Demo

This example will demonstrate how to instrument a complex function: a sort algorithm. This algorithm will order the items in an array according to a given criterion. In this case, the criterion is to reorder the numbers in `array` to ascending order. The question is: how efficient is this algorithm?

Solution

Possibly the simplest algorithm to order a list of values is the **Bubble Sort**. This algorithm will first find the largest item and put it at the end of the list. Then it will find the second largest and put it in the second-to-last spot (`iSpot`) and so on.

We will instrument this function with the `numCompare` variable. Initially set to zero, we will increment `numCompare` each time a pair of numbers (`array[iCheck] > array[iCheck + 1]`)is compared. For convenience, we will return this value to the caller.

```
/*****************************************************************
 * BUBBLE SORT
 * Instrumented version of the Bubble Sort. We will return the number
 *  of times elements in the array were compared.
 *****************************************************************/
int bubbleSort(int array[], int numElements)
{
   // number of comparisons is initially zero
   int numCompare = 0;

   // did we switch two values on the last time through the outer loop?
   bool switched = true;

   // for each spot in the array, find the item that goes there with iCheck
   for (int iSpot = numElements - 1; iSpot >= 1 && switched; iSpot--)
      for (int iCheck = 0, switched = false; iCheck <= iSpot - 1; iCheck++)
      {
         numCompare++;       // each time we are going to compare, add one

         if (array[iCheck] > array[iCheck + 1])
         {
            int temp = array[iCheck];     // swap 2 items if out of order
            array[iCheck] = array[iCheck + 1];
            array[iCheck  + 1] = temp;
            switched = true;              // a swap happened, do outer loop again
         }
      }

   return numCompare;
}
```

It is important to note that instrumenting should not alter the functionality of the program. Removing the instrumentation code should leave the algorithm unchanged.

See Also

The complete solution is available at 4-4-bubbleSort.cpp or:

```
/home/cs124/examples/4-4-bubbleSort.cpp
```

Example 4.4 – Instrument Fibonacci

Demo

This example will demonstrate how to instrument two functions computing the Fibonacci sequence. It will not only demonstrate how to add counters at performance-critical locations in the code, but will demonstrate how to surround the instrumentation code with `#ifdef`s enabling the programmer to conveniently remove the instrumentation code or quickly add it back.

Problem

As you may recall, the Fibonacci sequence is defined as the following:

$$F(n) := \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Write a program to compute the n[th] Fibonacci number.

```
How many Fibonacci numbers shall we identify? 10
Array method:     55
Recursive method: 55
```

In debug mode (compiled with the `-DDEBUG` switch) , the program will display the cost:

```
How many Fibonacci numbers shall we identify? 10
Number of iterations for the array method: 9
Number of iterations for the recursive method: 177
```

Solution

In order to not influence the normal operation of the Fibonacci functions, the following code was added to the top of the program:

```
#ifdef DEBUG          // all the code that will only execute if DEBUG is defined
int countIterations = 0;
#define increment()     countIterations++
#define reset()         countIterations = 0
#define getIterations() countIterations

#else // !DEBUG
#define increment()
#define reset()
#define getIterations()  0
#endif
```

Thus in ship mode (when `DEBUG` is not defined), the "functions" `increment()`, `reset()`, and `getIterations()` are defined as nothing. In debug mode, these manipulate the global variable `countIterations`. Normally global variables are dangerous. However, since this variable is only visible when `DEBUG` is defined, its damage potential is contained. The last thing to do is to carefully put `reset()` before we start counting and `getIterations()` when we are done counting.

```
   reset();
   int valueArray = computeFibonacciArray(num);
   int costArray  = getIterations();
```

This, coupled with `increment()` when counting constitutes all the instrumentation code in the program.

See Also

The complete solution is available at 4-4-fibonacci.cpp or:

```
/home/cs124/examples/4-4-fibonacci.cpp
```

Assignment 4.4

Our assignment this week is to determine the relative speed of a linear search versus a binary search using instrumentation. To do this, start with the code at:

```
/home/cs124/assignments/assign44.cpp
```

Here, you will need to modify the functions `binary()` and `linear()` to count the number of comparisons that are performed to find the target number.

Next, you will need to modify `computeAverageLinear()` and `computeAverageBinary()` to determine the number of compares on average it takes to find each element in the array.

Example

Consider the file `numbers.txt` that has the following values:

```
1 4 10 36 47 92 100 110 125 136 142 143 150 160 167
```

For the above example, the list is:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	4	10	36	47	92	100	110	125	136	142	143	150	160	167

When we run the program on the above list, the program will compute how long it takes to find an element in the list using the linear search method and using the binary search method. If I call `linear()` (a function performing a linear search from left to right) with the search parameter set to `4`, then I will find it with two comparisons. This means `linear(list, num, 4) == 2` because the function `linear()` will return the number of comparisions, `list` contains the list of numbers, and `num` is the number of items in `list`. Thus `linear(list, num, 47) == 5`. To find the average cost of the linear search, the equation will be:

```
(linear(list, num, 1) + linear(list, num, 4) + … + linear(list, num, 167)) / num ==
                          (1 + 2 + ... + 15) / 15 ==
                                 120 / 15 ==
                                    8.0
```

To compute the average cost of the binary search, the equation will be:

```
(binary(list, num, 1) + binary(list, num, 4) + … + binary(list, num, 167)) / num == 3.3
```

The user input is **underlined**.

```
Enter filename of list: numbers.txt
Linear search:        8.0
Binary search:        3.3
```

Assignment

The test bed is available at:

```
testBed cs124/assign44 assignment44.cpp
```

Don't forget to submit your assignment with the name "Assignment 44" in the header.

Please see page 358 for a hint.